PRODUCTION OPERATIONS MANUAL

Orion AWS Architecture

A complete map of the production digital-signage platform: every AWS resource, how traffic and data move, monthly cost in Mumbai (ap-south-1), and the build pipeline that runs on every push to main.

Region ap-south-1 (Mumbai)
CloudFormation stacks: Orion and Orion-github-oidc
Dashboard: Amplify Hosting Compute (orion-dashboard-ssr)
API: NestJS on EC2 Auto Scaling · Worker: SQS consumer
Database: RDS PostgreSQL 16 · Media: private S3 + signed CloudFront
Source of truth: infra/aws/cdk + .github/workflows/deploy-aws.yml

Account resources live in CloudFormation stack Orion
CI identity: stack Orion-github-oidc / role orion-github-deploy

17 September 2026
Version 1.0

Contents

1. Purpose and current posture
2. How the system is connected
3. Request paths (CMS, players, uploads)
4. Resource catalog and cost
5. Monthly cost model
6. Capacity, health, and scaling
7. Security and identity
8. Build and deploy flow on every push
9. What happens on a new EC2 box
10. Stack outputs and operations
11. Risks and recommended next changes

How to read cost numbers

Prices are on-demand USD in ap-south-1, using ~730 hours/month. They describe the current CDK sizes (API min 1 / max 2, worker min 1 / max 2, db.t4g.small, one NAT). Usage line items (CloudFront GB, S3 media, NAT GB, Amplify SSR) move with traffic. Raising API min to 2 adds about $9/month; it is called out separately and is not already applied in this document.

1. Purpose and current posture

Orion is a digital-signage control plane. Operators use a Next.js CMS to manage organizations, devices, playlists, and assets. Android players pair to the API, download media, play locally, send a heartbeat about every 60 seconds, and upload proof-of-play batches. Staging remains on Netlify (dashboard) and Render (API). AWS is production only, in a single account and a single region: Mumbai.

The production CloudFormation stack is named Orion. A smaller companion stack, Orion-github-oidc, holds the IAM role GitHub Actions assumes. There is no Kubernetes, no ECS service, and no custom domain required. Without DNS, AWS still publishes HTTPS URLs: Amplify for the CMS, CloudFront for the API and for signed media.

Fact	Value
Region	ap-south-1 (Mumbai)
Primary stack	Orion
Identity stack	Orion-github-oidc (role orion-github-deploy)
VPC	orion-vpc, 2 AZs, 1 NAT Gateway
CMS	Amplify Hosting Compute, app orion-dashboard-ssr, branch main
API compute	ASG orion-api, t4g.small ARM, Docker NestJS :3001, min 1 / max 2
Worker compute	ASG orion-worker, t4g.small ARM, min 1 / max 2
Database	RDS PostgreSQL 16, db.t4g.small, Single-AZ, 20 GB gp3, private
Object storage	S3 orion-media-<account>, Block Public Access, versioned
Player protocol	Heartbeat 60s, full sync fallback 600s, PoP ~5 min; no 5s revision poll

This size is a small production, not the original 2,000-device Multi-AZ plan. Playback does not stream through the API. Players fetch objects from CloudFront with signed URLs and play from local disk. The API is the control plane.

2. How the system is connected

Think of three planes that share one VPC and one database. The CMS plane is Amplify. The API plane is CloudFront to ALB to EC2. The media plane is S3 behind a second CloudFront distribution that requires a signed cookie/URL key group. The worker plane has no load balancer; it only consumes SQS and writes Postgres.

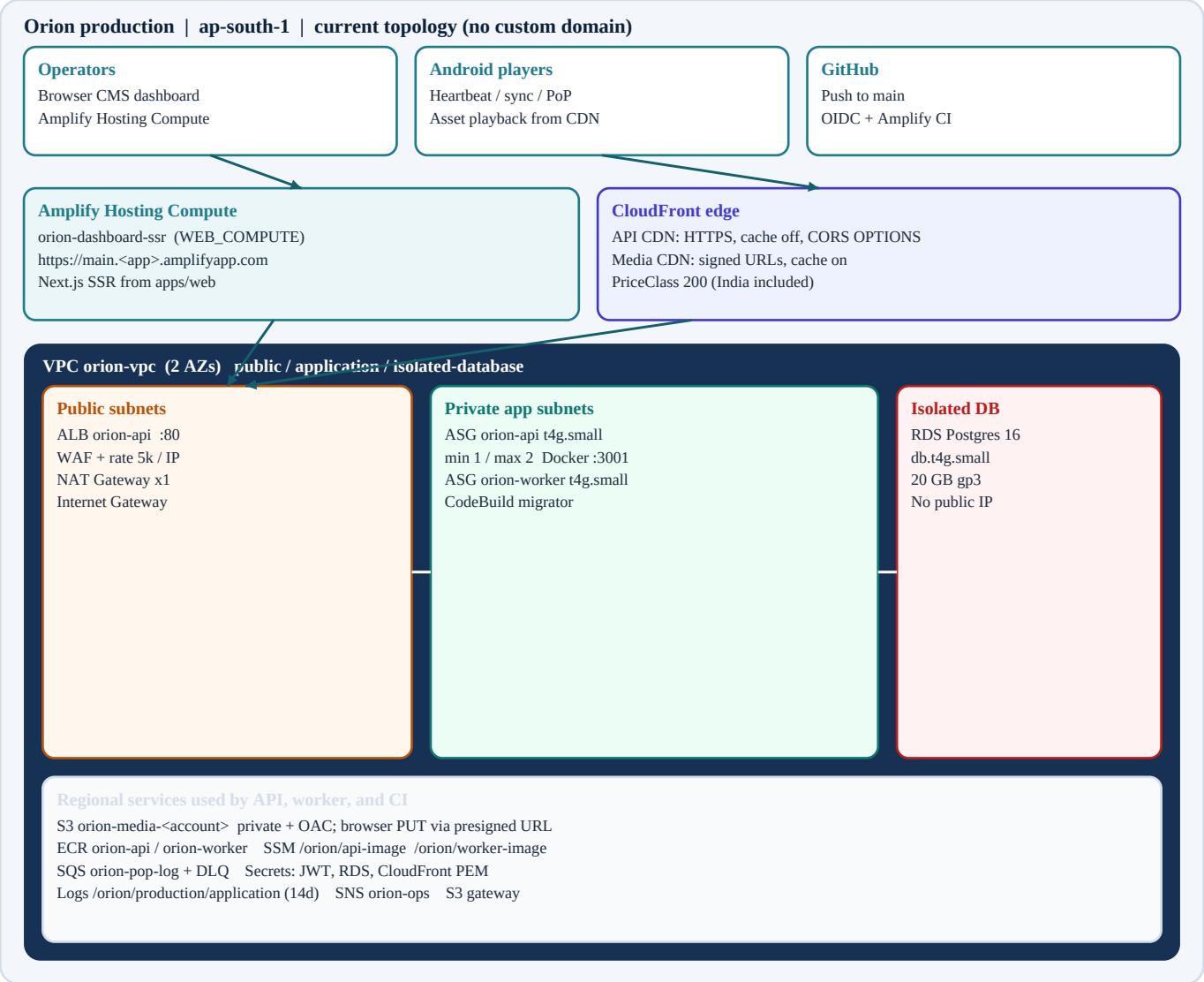


Figure 1. Logical wiring of clients, edge, VPC, and shared AWS services. Arrows on the diagram are request or control direction, not a complete security-group matrix.

Subnet and security-group wiring

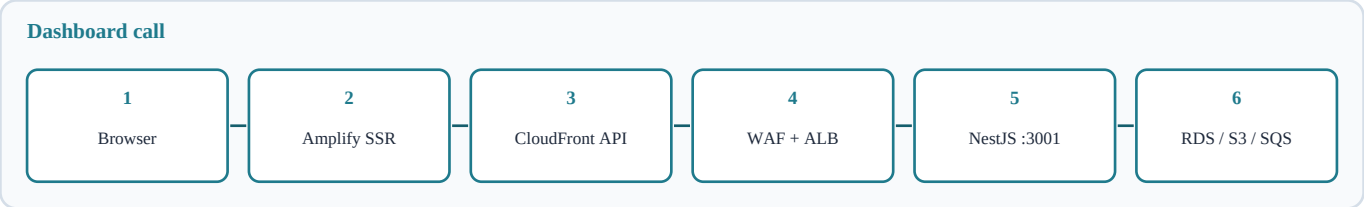
Layer	Where	Who can reach it
Public	2 public /24 subnets	Internet to ALB :80 (or :443 with a cert). NAT Gateway for private egress. IGW.
Application	2 private-with-egress /24	ALB security group to API :3001. Instances reach RDS :5432, S3 via gateway endpoint, and AWS APIs via the NAT.
Database	2 isolated /24	No NAT, no internet. Ingress 5432 only from API, worker, and CodeBuild migrator security groups.

IAM instance roles are separate. The API role can pull ECR orion-api, read/write the media bucket, send SQS, read JWT and CloudFront private-key secrets, read SSM /orion/api-image, and write CloudWatch. The worker role pulls orion-worker, consumes the PoP queue and DLQ, and reads the database secret. Neither role is publicly assumable.

3. Request paths

Three paths matter in production. They share Postgres and the media bucket, but they do not share compute. A CMS click never lands on the worker. A player heartbeat never lands on Amplify. An asset file never transits the NestJS process except as a presigned URL.

3.1 Operator using the CMS

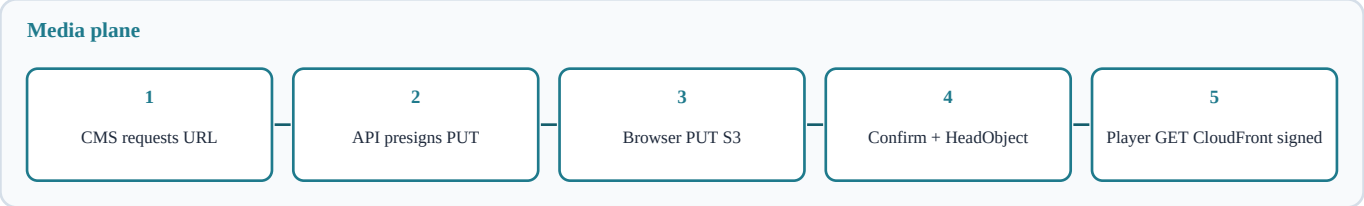


The browser loads the Next.js app from Amplify Hosting Compute. NEXT_PUBLIC_API_URL is baked in at Amplify build time from the stack ApiUrl output, so it is the CloudFront API domain today. JSON calls go browser to CloudFront (HTTPS, cache disabled). A CloudFront Function answers OPTIONS with 204 and CORS headers so a 502 origin cannot look like a CORS failure. Other methods go HTTP :80 to the internet-facing ALB, through the regional WAF (AWSManagedRulesCommonRuleSet plus 5,000 requests / 5 minutes / IP), then to healthy targets on :3001. NestJS applies CORS_ORIGINS from the Amplify default domain and the main branch URL.

3.2 Android player (steady state)

After pairing, the player authenticates with a device token, not a CMS JWT. Every ~60 seconds it POSTs /api/player/heartbeat. That call is not a cheap ping: it updates the Device row (presence, CPU, RAM, now-playing) and resolves schedule, content revision, and any pending cache command. About every 10 minutes it falls back to GET /api/player/sync for a full manifest. Proof-of-play posts to /api/player/pop-logs about every 5 minutes; the API validates and enqueues SQS orion-pop-log; the worker writes rows. Media bytes are not on this path. GET /api/player/sync-revision is advertised with interval 0 and must not be polled.

3.3 Asset upload and playback



The CMS asks the API for an upload URL. The API creates an asset row and returns a 15-minute S3 presigned PUT against orion-media-<account>. The browser uploads directly to S3 (this requires bucket CORS for the Amplify origin; that rule is not in CDK today and must be maintained on the bucket). After PUT, the CMS confirms; the API HeadObject-checks the key. Players later receive CloudFront signed GET URLs. The media distribution uses Origin Access Control, is not public, and trusts a CloudFront key group whose public key is a CDK parameter and whose private key lives in Secrets Manager orion/cloudfront-private-key.

Important split

CloudFront in front of the API exists only while there is no custom api. domain. After ROOT_DOMAIN and an ap-south-1 ACM certificate are set, clients should hit the ALB on HTTPS :443 and the API CloudFront distribution is not created. Media CloudFront always remains, with or without media.yourdomain.com.

4. Resource catalog and cost

The following is every durable resource the Orion stack creates, grouped the way the CDK constructs them. Hourly prices are on-demand in ap-south-1. 'Always-on' means the resource exists even with zero players. ECR, SQS, SSM, and most IAM are cents unless storage or request volume grows.

4.1 Networking

Resource	Config	Role	Always-on USD
VPC orion-vpc	2 AZs, 6 subnets /24	Isolation boundary	$0
Internet Gateway	1	Inbound to ALB, public routing	$0
NAT Gateway	One public subnet	ECR, Secrets, SSM, CloudWatch, SQS from private instances	$41 hourly + $0.056/GB
S3 gateway endpoint	1	Private S3 without NAT for Head/Get/Put from API	$0
Public IPv4	NAT EIP + ALB AZs	AWS $0.005/hr per address	~$11
Security groups	ALB, API, worker, RDS, migrator	Port 80/443 public; 3001 from ALB; 5432 from app	$0

4.2 Edge and CMS

Resource	Config	Role	Always-on USD
Amplify CfnApp	orion-dashboard-ssr, WEB_COMPUTE	Next.js CMS. Git-connected to this repo, auto-build on main.	SSR $0.30/M req + $0.20/GB-hour; builds $0.01/min
Amplify branch	main, framework Next.js - SSR	Production hostname https://main.<defaultDomain>	Included with app usage
Amplify IAM role	orion-amplify-hosting	AdministratorAccess-Amplify for Hosting Compute	$0
CloudFront API	PriceClass 200, cache off	HTTPS without a custom domain; Function for OPTIONS 204	Request + GB out (usage)
CF response headers	ApiCorsPolicy	Adds CORS on API responses	$0
CF Function	viewer-request preflight	Stops OPTIONS hitting a dead origin	~$0.10 per 1M invocations
CloudFront media	OAC + trusted key group, cache on	Signed playback/download of private S3 objects	Request + GB out (usage)
CF public key / key group	From CDK parameter	Verifies signed media URLs	$0

4.3 Compute and load balancing

Resource	Config	Role	Always-on USD
ALB orion-api	Internet-facing, idle 60s	Spreads HTTP to healthy API instances. Does not scale the ASG.	$20-25 hours+LCU
WAF WebACL	REGIONAL on the ALB	Common rule set + 5,000 req/5 min/IP	$5 ACL + ~$2 rules + $0.60/M req
Target group	/api/ready every 30s	2 OK = in; 3 fail = out; timeout 5s	$0
ASG orion-api	min 1, max 2, t4g.small	Runs Docker NestJS from /orion/api-image. CPU target 60%. ELB health, 10 min grace.	$8.18 per running instance
ASG orion-worker	min 1, max 2, t4g.small	SQS consumer. CPU 70% plus queue step scale at 200 / 2,000 messages.	$8.18 per running instance
Launch templates	AL2023 ARM, IMDSv2	User data installs Docker, CW agent, systemd unit	Included in EC2
EBS gp3 root	~8 GB each instance	OS + Docker layers	~$1 per volume
CodeBuild orion-migrate	ARM, VPC, on demand	docker run migrate-db.sh against RDS before rollout	Cents per deploy

4.4 Data and async

Resource	Config	Role	Always-on USD
RDS Postgres 16	db.t4g.small, Single-AZ, encrypted	System of record. PubliclyAccessible false. Backup 7 days. deletionProtection false.	~$31 compute + ~$3 storage
RDS secret	generated user orion	DATABASE_URL for API, worker, migrator	$0.40
S3 media bucket	orion-media-<account>	Private assets. Versioned. Abort MPU 7d. Noncurrent expire 30d. RETAIN on delete.	$0.023-0.03/GB-month + requests
SQS orion-pop-log	20s long poll, 120s vis.	Proof-of-play batches from API to worker	Near $0 at current volume
SQS orion-pop-log-dlq	14 day retention	After 5 receive failures	Near $0
Log group	/orion/production/application	Docker awslogs from API and worker, 14 day retain	Ingest + store (usage)
SNS orion-ops	RDS event subscription	DB failure/failover/backup/maintenance. Alarms exist but are not all wired to this topic.	$0 plus deliveries

4.5 Identity, images, and parameters

Resource	Config	Role	Always-on USD
ECR orion-api	scan on push, keep 40 images	linux/arm64 NestJS image	~$0.10/GB stored
ECR orion-worker	scan on push, keep 40 images	linux/arm64 worker image	~$0.10/GB stored
SSM /orion/api-image	string, git SHA tag	What the API systemd unit docker-pulls	$0
SSM /orion/worker-image	string, git SHA tag	What the worker unit docker-pulls	$0
Secret orion/jwt	64-char generated	CMS and API JWT signing	$0.40
Secret orion/cloudfront-private-key	PEM, uploaded once	Must match CloudFrontPublicKey parameter. Placeholder until put-secret-value.	$0.40
OIDC provider + role	orion-github-deploy	GitHub Actions on main / production env only. Can assume cdk-* and push ECR, SSM, refresh ASG, start CodeBuild.	$0
CloudWatch dashboard + alarms	CPU, mem, 5xx, RDS, SQS	Visibility. Not auto-remediation.	Dashboard/alarm free tier typically covers

4.6 What is intentionally not in AWS production

- No custom domain unless GitHub production variables ROOT_DOMAIN, API_CERTIFICATE_ARN, MEDIA_CERTIFICATE_ARN are set.
- No Multi-AZ RDS, no second NAT, no RDS Proxy, no ElastiCache.
- No ECS/EKS. One Docker container per EC2 instance via systemd.
- Staging is still Netlify + Render; this stack does not clone a staging VPC.

5. Monthly cost model

Use two numbers. The always-on floor is what you pay with the current min sizes even if nobody opens the CMS. The traffic band is extra CloudFront, NAT GB, S3, WAF requests, and Amplify SSR. The README figure of $70-90/month assumed 1 API + 1 worker and understated Mumbai NAT (~$41/month by itself).

Scenario	API instances	Always-on floor	Likely invoice
Current CDK (min 1)	1 API + 1 worker	~$140-155	$90-130 quiet; $150-170 typical
Recommended min 2 / max 4	2 API + 1 worker	~$150-165	About +$9 vs today
CPU scale to max 4 + 2 workers	4 API + 2 worker	~$175-190 while scaled	Only while CPU/queue stay high
Plus RDS Multi-AZ later	unchanged compute	+ ~$31 RDS	HA for database, not for API

NAT is the largest fixed line. The extra API instance is not. Media egress for a thousand players downloading new videos will dwarf EC2. Player heartbeats for 1,000 devices are about 43 million API requests/month through CloudFront and WAF; request charges then become visible even though EC2 CPU may still look idle.

Always-on component	Monthly USD	Notes
NAT Gateway hourly	41	One AZ. Does not include $0.056/GB processed.
RDS db.t4g.small + 20 GB	34	Single-AZ Postgres 16.
EC2 1x API + 1x worker t4g.small	16	On-demand $0.0112/hr each.
ALB hours + light LCU	20-25	LCU grows with connections and GB.
Public IPv4	11	NAT EIP plus ALB AZ addresses.
WAF ACL + rules	7-10	Plus $0.60 per million requests.
Secrets (3) + ECR/EBS/SQS	5-8	Grows with image and log volume.
Amplify + CloudFront at idle	2-10	CMS traffic only.

6. Capacity, health, and scaling

The load balancer never launches instances. Auto Scaling does, and only inside min/max. Today API min is 1, so a single t4g.small is a full CMS+player outage until a replacement passes /api/ready (often several minutes: Docker pull plus 10-minute ELB grace).

Signal	Policy	What it does
API CPU 60% average	Target tracking on ASG orion-api	Adds a second t4g.small; cannot exceed max 2. Scale-in when CPU drops.
Worker CPU 70%	Target tracking on orion-worker	Same pattern, max 2.
SQS visible messages	Step: +1 at 200, +2 at 2,000	Proof-of-play backlog. Capped by max 2.
API CPU 80% / mem 85%	CloudWatch alarm only	Does not scale. Page a human if SNS is subscribed.
Unhealthy hosts, ALB 5xx, p95 > 2s	Alarms	Observability, not capacity.
RDS CPU 80%, connections 60, free storage < 4 GB	Alarms	The database is the shared bottleneck for both API instances.

Rolling updates: API keeps minInstancesInService 1 and replaces 1 at a time. Worker may go to 0 during replace. GitHub instance refresh uses MinHealthyPercentage 100% for API (never drop below current healthy count during a rollout) and 50% for worker.

Steady state for 1,000 well-behaved players is about 17 heartbeat requests/s plus a few sync and PoP calls. That is easy for the ALB and plausible for two t4g.small processes, but each heartbeat writes Postgres. The RDS instance, not the ALB, is the first hard limit. A reconnect storm (all devices GET /sync at once) will not wait for Auto Scaling; new instances take minutes.

7. Security and identity

- S3 Block Public Access on the media bucket. CloudFront reads via OAC; browsers write only with a short-lived presigned PUT.
- RDS has no public IP. Security group allows 5432 only from API, worker, and migrator.
- IMDSv2 required on launch templates.
- WAF common rules plus per-IP rate limit on the ALB. Shared NAT at a venue can trip 5,000/5 min.
- GitHub OIDC: role orion-github-deploy only from this repo's main branch or production environment. No long-lived AWS keys in Actions.
- Amplify GitHub App token is a CDK parameter (AMPLIFY_GITHUB_TOKEN). CloudFront public PEM is a no-echo parameter.
- Player routes are not JwtAuthGuard; they use the device token issued at pairing.
- S3 bucket CORS is required for CMS uploads and is not declared on the Bucket construct. A console/CLI CORS rule can disappear conceptually whenever someone recreates the bucket; it should be added to CDK.

8. Build and deploy flow on every push

A push to main starts two independent pipelines. GitHub Actions deploys infrastructure and rolls API/worker images. Amplify, watching the same commit through its GitHub App connection, builds the CMS. They do not wait for each other. The CMS can go live on an old ApiUrl, and the API can go live while Amplify is still building.

8.1 Triggers and identity

Pipeline	Trigger	Identity	What it ships
GitHub Actions: Deploy Orion production	push to main, or workflow_dispatch	OIDC to orion-github-deploy (env production)	CDK stack, ARM images, migrations, ASG refresh
Amplify Hosting Compute	autoBuild on branch main	Amplify service role orion-amplify-hosting	Next.js SSR bundle from apps/web

Concurrency group orion-aws-production with cancel-in-progress false: a second push queues; it does not cancel an in-flight deploy. Region is always ap-south-1. Required GitHub environment production: variable AWS_DEPLOY_ROLE_ARN; secrets CLOUDFRONT_PUBLIC_KEY and AMPLIFY_GITHUB_TOKEN. Optional: ROOT_DOMAIN, API_CERTIFICATE_ARN, MEDIA_CERTIFICATE_ARN.

8.2 GitHub Actions steps (in order)

Step	What runs	Why it exists
1. checkout	actions/checkout@v4	Monorepo at the git SHA being deployed.
2. AWS credentials	configure-aws-credentials@v4, role-to-assume AWS_DEPLOY_ROLE_ARN	Short-lived STS creds. No access keys in the repo.
3. recover leftovers	infra/aws/scripts/recover-failed-stack.sh	After a rolled-back deploy, named resources (ECR, S3, Amplify, log groups) can block CloudFormation. Script deletes or imports them.
4. Node 22 + npm ci	prisma generate, build:api, build:worker	Compile check on the runner before images are built.
5. CDK install	infra/aws/cdk npm install	Separate package graph from the monorepo.
6. cdk deploy Orion	require-approval never, import-existing-resources	Creates/updates VPC, ALB, ASGs, RDS, Amplify app config, WAF, queues, secrets. Passes CloudFront PEM and GitHub token as parameters.
7. docker buildx	linux/arm64 Dockerfile.api and Dockerfile.worker	QEMU on ubuntu-latest. Tags: ECR URI:git SHA. Pushed immutably.
8. CodeBuild migrate	orion-migrate with IMAGE_URI=api:SHA	Inside the VPC, docker run bash scripts/migrate-db.sh. API has SKIP_STARTUP_MIGRATE=true so instances never migrate on boot. Poll up to ~20 minutes.
9. Promote + refresh	SSM put-parameter then StartInstanceRefresh	SSM /orion/api-image and /orion/worker-image become the new SHA. ASG launches new boxes that pull that URI.

8.3 Docker images

Dockerfile.api is a two-stage Node 22 build: npm ci, prisma generate, npm run build:api, then a slim runtime with openssl, the dist tree, prisma schema, start-api.sh, and migrate-db.sh. Port 3001. Dockerfile.worker is the same pattern without an HTTP port. Both are linux/arm64 to match t4g.small Graviton. ECR lifecycle keeps 40 images.

8.4 Amplify CMS build (parallel, not in Actions)

CDK writes the Amplify buildSpec (mirrored in repo amplify.yml). appRoot is apps/web but buildPath is repo root so npm ci sees the workspace lockfile. AMPLIFY_MONOREPO_APP_ROOT is apps/web. NEXT_PUBLIC_API_URL is the stack ApiUrl. NODE_VERSION is 22.

Phase	Commands	Intent
preBuild	nvm use 22; npm ci; prepare-amplify-compute.sh pre	Install workspaces. Copy a real next package (not an npm-workspace symlink) so next/dist/server/next.js exists in the compute bundle.
build	npm run build:amplify --workspace @orion/web; prepare-amplify-compute.sh post	next build --webpack (not Turbopack). Standalone output. Materialize next into .next/standalone.
artifacts	baseDirectory apps/web/.next	Amplify Hosting Compute serves SSR from this bundle.
cache	node_modules and apps/web/.next/cache	Faster subsequent builds.

Turbopack and workspace symlinks previously shipped a compute bundle whose next package.json existed but dist/server/next.js did not. The prepare script is mandatory. Wait until the Amplify console job for main is SUCCEED before treating AppUrl as live. Changing the API URL requires a new Amplify build because NEXT_PUBLIC_API_URL is inlined.

Order of safety

Migrations run after images are in ECR and before instance refresh. If migrate fails, Actions stops; old ASG instances keep serving the previous image. If CDK succeeds but the image build fails, infrastructure may have changed while compute still runs the last good SHA in SSM.

9. What happens on a new EC2 box

User data (Amazon Linux 2023 ARM) is the contract between CDK and runtime. A brand-new instance, or one launched by instance refresh, does all of this before it can pass ELB health.

Order	Action
1	dnf install docker, amazon-cloudwatch-agent, python3; enable docker.
2	Write CloudWatch agent config (mem_used_percent, disk used_percent, dimension AutoScalingGroupName).
3	Write /opt/orion-api.sh or orion-worker.sh.
4	Script reads SSM image URI, Secrets Manager DB secret (and JWT / CloudFront PEM on API), builds DATABASE_URL.
5	Writes /etc/orion/service.env (S3_BUCKET, CORS_ORIGINS, CLOUDFRONT_*, SQS URL, NODE_ENV=production, ...).
6	docker login to ECR, pull URI (retry up to 30 x 20s), rm old container, docker run --env-file with awslogs to /orion/production/application.
7	systemd enable --now orion-api or orion-worker, Restart=always.
8	API publishes :3001. ALB needs two successes of GET /api/ready. Grace period 10 minutes.

That is why scale-out is slow compared with Lambda, and why min 2 is the real high-availability control: the second box must already be InService before the first one disappears.

10. Stack outputs and operations

CloudFormation stack Orion publishes the URLs and names the pipelines consume:

Output	Meaning
AppUrl / AmplifySsrAppUrl	CMS. With no domain: https://main.<amplify default domain>
ApiUrl	Public API HTTPS (CloudFront domain until custom api. DNS exists)
MediaUrl	Signed media CloudFront (or media.domain later)
ApiDnsTarget / MediaDnsTarget	What to CNAME when a domain is added
AmplifySsrAppId	Console deep link for the Compute app
ApiRepositoryUri / WorkerRepositoryUri	ECR push targets
MigrationProjectName	orion-migrate CodeBuild project
ApiAutoScalingGroupName / WorkerAutoScalingGroupName	orion-api / orion-worker
ApiImageParameterName / WorkerImageParameterName	/orion/api-image and /orion/worker-image
ApplicationLogGroupName	/orion/production/application
OpsTopicArn	Subscribe email/Slack here
CloudFrontPrivateKeySecretArn	Put the PEM once after first deploy, then instance-refresh API

Day-2 commands (from the AWS README)

After first deploy, upload the CloudFront private key: aws secretsmanager put-secret-value --region ap-south-1 --secret-id orion/cloudfront-private-key --secret-string file://cloudfront-private-key.pem. Then aws autoscaling start-instance-refresh --auto-scaling-group-name orion-api --preferences MinHealthyPercentage=100,InstanceWarmup=300 so running boxes load the PEM.

Create the first admin with POST $API_URL/api/auth/bootstrap/super-admin. Point Android players at ApiUrl. Rollback: write a previous image URI into /orion/api-image and /orion/worker-image, then start instance refreshes. Do not rely on docker tag latest.

11. Risks and recommended next changes

Risk	Why it matters	Practical change
API min 1	One instance death or deploy is a full API outage for every player.	Set ASG orion-api min 2, max 4 in CDK and console. ~+$9/month.
Single-AZ db.t4g.small	Patching or AZ issue takes CMS and players down. Heartbeat writes contend here first.	Move to db.t4g.medium; add Multi-AZ when downtime is unacceptable. Enable deletionProtection.
S3 CORS not in CDK	CMS uploads fail with browser preflight errors after a bucket recreate or origin change.	Declare cors: on the media Bucket with the Amplify origins plus localhost.
WAF 5,000 / IP	A mall or hotel NAT in front of many players can look like one client.	Raise the rate limit if devices share egress IPs.
Alarms without subscribers	CPU/5xx/RDS alarms fire silently.	Subscribe orion-ops to email or Slack.
Amplify vs API race	NEXT_PUBLIC_API_URL can lag a CDK URL change.	Redeploy the Amplify version after any ApiUrl change.
NAT x1	ECR pulls and secret reads fail if that NAT dies; existing processes may keep running.	Second NAT is ~+$41/month; optional until multi-AZ is a goal.

Document control

This PDF describes the repository as of 17 September 2026: CDK OrionEnvironmentStack, workflow deploy-aws.yml, amplify.yml, and Dockerfiles under infra/aws/docker. Invoice totals always win over the model in section 5. Re-generate after material CDK changes.

End of document. For deploy runbooks see infra/aws/README.md. For player protocol details see ANDROID_PLAYER_PROMPT.md. Do not treat the original 2,000-device Multi-AZ plan as what is running today.